ARM Tutorial Solutions:

1.

(i)**MOV r7, r5, LSL #3**

PRE

- r5 = 5
- r7 = 8

Instruction:

MOV r7, r5, LSL #3

- LSL #3 means left shift r5 by 3 bits: 5<<3=40

POST

- r5 = 5 [unchanged]
- r7 = 40

MOVS r0, r1, LSL #1

PRE

- cpsr = nzcvqiFt_USER
- r0 = 0x00000000
- r1 = 0x80000004

Instruction:

MOVS r0, r1, LSL #1

- r1 = 0x80000004
- r1 << 1 = 0x00000008 (ignoring carry out) = 0x00000008

But, shifting 0x80000004 left by 1:

- Binary: 1000...00000000000000000100
- Shift left by 1: 0000...00000000000000001000 (0x00000008)
- The top bit (bit 31) was '1', so N=0
- The result is not zero, so Z=0
- No carry out (bit shifted out is '1'), so C=1
- Overflow is unaffected for MOV.

POST
r0 = `0x00000008`

cpsr = `n=0, z=0, c=1, v=unchanged`

MULL r0, r1, r2, r3

UMULL = Unsigned Multiply Long.
Multiplies two 32-bit registers → produces a 64-bit result.
Stores lower 32 bits into first destination (`r0`) and upper 32 bits into second destination (`r1`).

Step 1: PRE values

r0 = `0x00000000`
r1 = `0x00000000`
r2 = `0xF0000003`
r3 = `0x00000008`

Step 2: Convert operands

r2 = `0xF0000003` = unsigned decimal = `4026531843`
r3 = `0x00000008` = decimal = 8

Step 3: Multiply

4026531843×8=322122547444026531843 \times 8 =
32212254744 4026531843×8=32212254744

In hex:
`32212254744 = 0x780000018`

Step 4: Split into 64-bit result

64-bit result = `0x0000000780000018`

Lower 32 bits → `0x80000018`
Upper 32 bits → `0x00000007`

POST values:

r0 = `0x80000018`
r1 = `0x00000007`

LDR Variants (memory load addressing modes)

PRE:

- r0 = 0x00000000
- r1 = 0x80000000
- mem32[0x00009000] = 0x01010111
- mem32[0x00009004] = 0x01234567

The value of r1 should have been 0x0009000, in order to answer the question. With r1 = 0x80000000, the values of mem32[0x80000000], and mem32[0x80000004] are unknown. The following solution assumes that r1 is 0x0009000.

a) LDR r0, [r1, #4]

```
LDR r0, [r1, #4]
```

Address = r1 + 4 = `0x00009000 + 4 = 0x00009004`
Load r0 = mem32[0x00009004] = `0x01234567`
r1 is unchanged

Post:

r0 = `0x01234567`
r1 = `0x00009000`

b) LDR r0, [r1, #4]!

Address = r1 + 4 = `0x00009004`
r0 = mem32[0x00009004] = `0x01234567`
r1 = updated → `0x00009004`

Post:

r0 = `0x01234567`
r1 = `0x00009004`

c) LDR r0, [r1], #4
Address = r1 = `0x00009000`
r0 = mem32[0x00009000] = `0x01010111`
r1 = updated after → `0x00009000 + 4 = 0x00009004`
Post:

r0 = `0x01010111`
r1 = `0x00009004`

STMIB / LDMDA Example
After executing the instructions:

- Initial values:
  - r0 = 0x00009000
  - r1 = 0x00000003
  - r2 = 0x00000004
  - r3 = 0x00000005
- After `STMIB r0!, {r1-r3}`:
  - r0 is updated to 0x0000900C
  - r1, r2, r3 remain unchanged
  - Memory values:
    - 0x9004: 0x00000003
    - 0x9008: 0x00000004
    - 0x900C: 0x00000005
- After `LDMDA r0!, {r4-r6}`:
  - r4 is loaded from 0x900C → 0x00000005
  - r5 is loaded from 0x9008 → 0x00000004
  - r6 is loaded from 0x9004 → 0x00000003
  - r0 is updated to 0x00009000
  - r1, r2, r3 remain unchanged

Final register contents:

- r0 = 0x00009000
- r1 = 0x00000003
- r2 = 0x00000004
- r3 = 0x00000005
- r4 = 0x00000005
- r5 = 0x00000004
- r6 = 0x00000003

2.	R4 5, 9, e,17, R5 4, 5,9,e, R6 3, 2,1,0 R7 5, 9,e,
3.	MOV R1,#0x00000004 MOV R0,#0x00000003 MOV R2,#0x00000000 B here AND R3,R0,#0x00000001 CMP R3,#0x00000000 BEQ 0x0000020C ADD R2,R2,R1 MOV R1,R1,LSL #1 MOV R0,R0,ASR #1 here CMP R0,#0x00000000 BNE 0x000001FC stop B stop
4.	R0 20, 21, 22, 23, 24 R1 1, 2, 3, 4 R2 −2, −1, 0, 1
5.	AREA CountOnes, CODE, READONLY ; Define the code section ENTRY ; Mark the entry point mov r1,#3 start MOV R3, #0 ; Clear R3 (the result register, initialized to 0) count_loop TST R1, #1 ; Test the least significant bit of R1 ADDNE R3, R3, #1 ; If the bit is 1 (NE = Not Equal), increment R3 LSR R1, R1, #1 ; Logical Shift Right R1 by 1 (move the next bit to the LSB) CMP R1, #0 ; Check if R1 is now 0 BNE count_loop ; If R1 is not 0, continue looping halt B halt ; Infinite loop to end the program (halt) END

6.	MOV R1,R0 MOV R2,#0x00000000 MOV R0,#0x00000000 B here label LDR R3,[R1,R0,LSL #2] ADD R2,R2,R3 ADD R0,R0,#0x00000001 here CMP R0,#0x00000064 BLT label BX R14
7.	R0-30, 8,3 R1-5 R2 -FFFFFFFF,0,1,2,3,4,3,2,1,0,FFFFFFFF,0,FFFFFFFF
8.	R1-0, 3,10, R2-30,3C, R3 -3,D